

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Яндекс.Контеста №2
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

Е. Коровы в стойла	3
Условие:	3
Решение:	4
О решении:	5
Ф. Число	6
Условие:	6
Решение:	7
О решении:	7
Г. Кошмар в замке	9
Условие:	9
Решение:	10
О решении:	11
Н. Магазин	12
Условие:	12
Решение:	13
О решении:	14

Е. Коровы в стойла

Условие:

На прямой расположены стойла, в которые необходимо расставить коров так, чтобы минимальное расстояние между коровами было как можно больше.

Формат ввода

В первой строке вводятся числа N ($2 < N \leq 10^5$) – количество стойл и K ($1 < K < N$) – количество коров. Во второй строке задаются N натуральных чисел в порядке возрастания – координаты стойл (координаты не превосходят 10^9).

Формат вывода

Выведите одно число – наибольшее возможное допустимое расстояние.

Пример 1

Ввод 

```
6 3
2 5 7 11 15 20
```

Вывод 

```
9
```

Пример 2

Ввод 

```
5 3
1 2 3 100 1000
```

Вывод 

```
99
```

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

bool can_place(long long diff, const vector<int>& coords, int traps_count, int cows_count) {
    int placed = 1;
    long long last_cow_coord = coords[0];

    for (int i = 1; i < traps_count; i++) {
        if (coords[i] - last_cow_coord >= diff) {
            last_cow_coord = coords[i];
            placed++;
            if (placed == cows_count) {
                return true;
            }
        }
    }

    return false;
}

int main() {
    int traps_count, cows_count;
    cin >> traps_count >> cows_count;

    vector<int> coords;
    for (int i = 0; i < traps_count; i++) {
        int trap_coord;
        cin >> trap_coord;
        coords.push_back(trap_coord);
    }

    long long left = 0;
    long long right = coords[traps_count - 1] - coords[0];

    while (left < right) {
        long long mid = (left + right + 1) / 2;
        if (can_place(mid, coords, traps_count, cows_count)) {
            left = mid;
        } else {
            right = mid - 1;
        }
    }

    cout << left;
}
```

О решении:

Идея:

Воспользуемся бинарным поиском по ответу. Представим, что у нас есть булева функция, принимающая значения максимального расстояния между стойлами на вход, а в ответ возвращающая значение true/false. Тогда очевидно, что до какого-то значения включительно функция принимает значения true, после чего начинает принимать значения false.

Воспользовавшись бинарным поиском, выясним, какое максимальное расстояние может быть между стойлами.

Сложность по времени:

$O(\text{traps_count} \log R)$ – так как бинарный поиск по результату у нас за $\log R$, где R – размах между первым стойлом и вторым, а совершаем бинарный поиск мы в худшем случае n раз, потому что функция проверки расстановки работает за $O(n)$.

Сложность по памяти:

$O(\text{traps_count})$ – так как храним массив длины количества стойл для бинарного поиска.

Г. Число

Условие:

Вася написал на длинной полоске бумаги большое число и решил похвастаться своему старшему брату Пете этим достижением. Но только он вышел из комнаты, чтобы позвать брата, как его сестра Катя вбежала в комнату и разрежала полосу бумаги на несколько частей. В результате на каждой части оказалось одна или несколько идущих подряд цифр.

Теперь Вася не может вспомнить, какое именно число он написал. Только помнит, что оно было очень большое. Чтобы утешить младшего брата, Петя решил выяснить, какое максимальное число могло быть написано на полоске бумаги перед разрезанием. Помогите ему!

Формат ввода

Входной файл содержит одну или более строк, каждая из которых содержит последовательность цифр. Количество строк во входном файле не превышает 100, каждая строка содержит от 1 до 100 цифр. Гарантируется, что хотя бы в одной строке первая цифра отлична от нуля.

Формат вывода

Выведите в выходной файл одну строку — максимальное число, которое могло быть написано на полоске перед разрезанием.

Пример 1

Ввод 

```
2
20
004
66
```

Вывод 

```
66220004
```

Пример 2

Ввод 

```
3
```

Вывод 

```
3
```

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

bool compare(const string& a, const string& b) {
    return a + b > b + a;
}

void sort_after_new_val(vector<string>& nums) {
    int iters = (int)nums.size() - 1;
    while (iters > 0 && compare(nums[iters], nums[iters - 1])) {
        swap(nums[iters], nums[iters - 1]);
        iters--;
    }
}

int main() {
    vector<string> nums;

    string num;
    while (cin >> num) {
        nums.push_back(num);
        sort_after_new_val(nums);
    }

    for (const string& num : nums) {
        cout << num;
    }
}
```

О решении:

Идея:

Воспользуемся сортировкой кусков не по абсолютному значению, а по тому,

в каком порядке склеенное число получается больше. Будем сортировать массив после каждого нового полученного числа. Тогда перед каждым новым числом у нас будет правильно отсортированный массив. В таком случае нам достаточно будет просто поставить новое число на свое место

Сложность по времени:

$O(n^2)$ – потому что n раз (количество кусков числа) в худшем случае переставляем кусок числа за n итераций.

Сложность по памяти:

$O(n)$ – потому что храним массив длины n .

Г. Кошмар в замке

Условие:

Ходят легенды, что пока Аврора спала, ей снилось, что она ходит по разным местам: леса, поля, города и сёла. И вот однажды она наткнулась на пещеру, в которой сидел мудрец. Когда мудрец поднял на Аврору глаза, он изрёк: «Дорогая Аврора! Ты уже годами скитаешься по этим землям. Я хочу предложить тебе задачу. Вот тебе строка s . Каждая буква из алфавита имеет свой вес c_i . Вес строки, которую ты можешь получить из s многократным обменом любых двух букв, вычисляется так: для каждой буквы алфавита посчитай максимальное расстояние между позициями, в которых стоит эта буква и перемножь его с весом этой буквы. Принеси мне строку максимально возможного веса, и я тебе расскажу, в чём смысл жизни».

К счастью, когда Аврора уже шла со строкой к мудрецу, её поцеловал Филипп, и Аврора вышла из этого кошмара. Теперь вам предлагается самим окунуться в этот кошмар и решить поставленную задачу.

Формат ввода

Дана строка, состоящая из строчных букв латинского алфавита ($1 \leq |s| \leq 10^5$). Следующая строка ввода содержит 26 чисел — веса букв латинского алфавита от «a» до «z», веса неотрицательны и не превосходят $2^{31} - 1$.

Формат вывода

Выведите строку s , в которой переставлены буквы так, чтобы полученный вес был максимально возможным. Если искомым вариантов несколько, выведите любой из них.

Пример

Ввод

```
lkshrules  
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
```

Вывод

```
slkhruels
```

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

void sort_by_weights(vector<int>& letters_more_than_one, vector<long long>& weights) {
    int len = (int)letters_more_than_one.size();
    for (int i = 0; i < len - 1; i++) {
        for (int j = 0; j < len - 1 - i; j++) {
            if (weights[letters_more_than_one[j + 1]] > weights[letters_more_than_one[j]]) {
                swap(letters_more_than_one[j], letters_more_than_one[j + 1]);
            }
        }
    }
}

int main() {
    string line;
    cin >> line;

    vector<long long> weights(26);
    for (int i = 0; i < 26; i++) {
        cin >> weights[i];
    }

    vector<int> letters_counts(26, 0);
    for (char ch : line) {
        letters_counts[ch - 'a']++;
    }

    vector<int> letters_more_than_one;
    for (int i = 0; i < 26; i++) {
        if (letters_counts[i] > 1) {
            letters_more_than_one.push_back(i);
        }
    }

    sort_by_weights(letters_more_than_one, weights);

    int line_len = (int)line.size();
    int left = 0;
    int right = line_len - 1;
    string sorted_line(line_len, '0');

    for (int shift : letters_more_than_one) {
        if (left == right) {
            break;
        }

        sorted_line[left] = char('a' + shift);
        left++;
    }
}
```

```

        sorted_line[right] = char('a' + shift);
        right--;
        letters_counts[shift] -= 2;
    }

    for (int i = 0; i < 26; i++) {
        while (letters_counts[i] > 0) {
            while (left <= right && sorted_line[left] != '0') {
                left++;
            }

            if (left > right) {
                break;
            }

            sorted_line[left] = char('a' + i);
            letters_counts[i]--;
        }
    }

    cout << sorted_line;
}

```

О решении:

Идея:

В конце строки вставляем буквы, которые имеют хотя бы одного двойника с максимальным весом, таким образом вклад этой буквы будет максимален, так как расстояние также максимально. Затем внутрь ставим новую пар с весом меньше. Так продолжаем до тех пор, пока не закончатся буквы с двойниками. После этого между пар ставим оставшиеся буквы. Таким образом увеличивая расстояния между парами.

Сложность по времени:

$O(n \log n)$ — так как совершаем $\log n$ итераций, чтобы расставить буквы.

Сложность по памяти:

$O(n)$ — потому что храним в памяти строку, полученную на вход.

Н. Магазин

Условие:

У Билла большая семья: трое сыновей, девять внуков. И всех надо кормить. Поэтому Билл раз в неделю ходит в магазин.

Однажды Билл пришел в магазин и увидел, что в магазине проводится акция под названием «каждый k -й товар бесплатно». Изучив правила акции, Билл выяснил следующее. Пробив на кассе товары, покупатель получает чек. Пусть в чеке n товаров, тогда n/k округленное вниз самых дешевых из них достаются бесплатно.

Например, если в чеке пять товаров за 200, 100, 1000, 400 и 100 рублей, соответственно, и $k = 2$, то бесплатно достаются оба товара по 100 рублей, всего покупатель должен заплатить 1600 рублей.

Билл уже выбрал товары, и направился к кассе, когда сообразил, что товары, которые он хочет купить, можно разбить на несколько чеков, и благодаря этому потратить меньше денег.

Помогите Биллу выяснить, какую минимальную сумму он сможет заплатить за выбранные товары, возможно разбив их на несколько чеков.

Формат ввода

Первая строка входного файла содержит два целых числа n, k ($1 \leq n \leq 100\,000, 2 \leq k \leq 100$) — количество товаров, которые хочет купит Билл и параметр акции «каждый k -й товар бесплатно».

Следующая строка содержит n целых чисел a_i ($1 \leq a_i \leq 10\,000$) — цены товаров, которые покупает Билл.

Пример

Ввод 

```
5 2
200 100 1000 400 100
```

Вывод 

```
1300
```

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int count, free;
    cin >> count >> free;

    vector<int> count_prices(10001, 0);
    for (int i = 0; i < count; i++) {
        int price;
        cin >> price;
        count_prices[price]++;
    }

    long long total_sum = 0, iter = 0;
    for (int i = 10000; i > 0; i--) {
        while (count_prices[i] > 0) {
            iter++;
            if (iter % free != 0) {
                total_sum += i;
            }

            count_prices[i]--;
        }
    }

    cout << total_sum;
}
```

О решении:

Идея:

Вместо того, чтобы запоминать цены товаров (≤ 10000) будем запоминать, сколько раз такая цена встретился в корзине. Количество товаров кладем в ячейку с индексом, совпадающим с суммой товара. Потом просто проходимся по списку от самых больших значений, не включая в итоговую стоимость каждый бесплатный товар. Сумма покупки здесь максимизируется, так как если идти по отсортированному по убыванию массиву, то мы будем максимизировать каждый k -ый бесплатный товар, который мы игнорируем.

Сложность по времени:

$O(\text{count} + 10000)$ – считываем данные + проходимся по массиву линейно.

Сложность по памяти:

$O(10000)$ – храним массив встречающихся цен.